Lesson 3 of 5

Food

Random food appears and the snake eats it.

Goal	A red dot appears at a random spot, and the snake eats it.
Concepts	random, distance(), constants, writing a helper function.
Starting point	Lesson 2's finished code. The complete Lesson 3 code is on the last page for anyone who needs to catch up.
Teacher prep	Open a Python shell to demo random.randrange.

Session plan (30 minutes)

Time	Activity
0-5	Run Lesson 2. Ask: what is the snake missing? (Something to eat.)
5-10	Shell demo: random.randrange(-280, 280, 20), run it several times.
10-20	Students add the three pieces below and test.
20-30	Discuss distance(), do a challenge, wrap-up.

Step 1: import random

Add this at the very top, under import turtle:

```
import random
```

Step 2: create the food

Add this **after the head setup**:

```
food = turtle.Turtle("circle")
food.color("red")
food.penup()
food.goto(0, 100)
```

Step 3: a function to move the food

Add this **above game_loop**:

```
def move_food():
    food.goto(random.randrange(-LIMIT, LIMIT, STEP),
              random.randrange(-LIMIT, LIMIT, STEP))
```

Step 4: eat the food

Add this **inside game_loop, before move_head()**:

```
    if head.distance(food) < STEP:
        move_food()
```

Teaching notes

Show randrange in the shell first. The third number (the step) keeps the food lined up with the grid the snake moves on. Ask: why **distance(food) < STEP** and not **==**? Positions rarely match exactly, so we ask "close enough?" instead.

Challenge

- Change the food's colour and shape.
- Make the food only ever appear in the top half of the screen.
- Stretch: change the eating distance to 40. How does it feel?

Common problems

What you see	Likely cause and fix
NameError: name 'random' is not defined	import random is missing at the top.
Food never moves	The if block is not inside game_loop, or it is indented wrongly.
Food appears between grid squares	The third number (STEP) is missing from randrange.
Food is eaten from too far away	The distance number is too big. STEP (20) is about one square.

What we learned (student checklist)

[ ] I can generate random numbers.

[ ] I can measure how close two turtles are.

[ ] I can explain why we use < instead of == for collisions.

[ ] I can write a small helper function and call it from the game loop.

Complete Lesson 3 code (for catching up)

```
import turtle
import random

STEP = 20
LIMIT = 280

screen = turtle.Screen()
screen.title("Snake")
screen.bgcolor("black")
screen.setup(600, 600)
screen.tracer(0)

head = turtle.Turtle("square")
head.color("lime")
head.penup()
head.direction = "stop"

food = turtle.Turtle("circle")
food.color("red")
food.penup()
food.goto(0, 100)

def go_up():    head.direction = "up"
def go_down():  head.direction = "down"
def go_left():  head.direction = "left"
def go_right(): head.direction = "right"

def move_head():
    if head.direction == "up":    head.sety(head.ycor() + STEP)
    if head.direction == "down":  head.sety(head.ycor() - STEP)
    if head.direction == "left":  head.setx(head.xcor() - STEP)
    if head.direction == "right": head.setx(head.xcor() + STEP)

def move_food():
    food.goto(random.randrange(-LIMIT, LIMIT, STEP),
              random.randrange(-LIMIT, LIMIT, STEP))

def game_loop():
    if abs(head.xcor()) > LIMIT or abs(head.ycor()) > LIMIT:
        head.goto(0, 0)
        head.direction = "stop"

    if head.distance(food) < STEP:
        move_food()

    move_head()
    screen.update()
    screen.ontimer(game_loop, 100)

screen.listen()
screen.onkeypress(go_up, "Up")
screen.onkeypress(go_down, "Down")
screen.onkeypress(go_left, "Left")
screen.onkeypress(go_right, "Right")

game_loop()
screen.mainloop()
```